

Void Vanguard

Tartalomjegyzék

Fejlesztői dokumentáció	5
1. A játék alapjai	6
1.1. A szimuláció frissítési üteme	6
1.2. Animálás böngészőben	6
2. Merev testek fizikai jellemzői	7
2.1. Lineáris tehetetlenség	7
2.2. Tömegközéppont	7
2.3. Tehetetlenségi nyomaték	7
2.3.1. Párhuzamos tengely tétel	8
2.3.2. Tetszőleges sokszög tehetetlenségi nyomatéka	8
2.3.3. Összetett test tehetetlenségi nyomatéka	9
3. Játékmechanika	10
3.1. Rakétahajtóművek	10
3.1.1. Lineáris gyorsulás	10
3.1.2. Tolóerő	10
3.1.3. Forgatónyomaték	11
3.1.4. Szöggyorsulás	11
3.1.5. Szögsebesség	12
3.2. Lövés	12
3.2.1. Játékos lövésmechanikája	12
3.2.2. Ellenségek lövésmechanikája	12
3.2.3. Előtartás	12
3.3. Játékosmodell módosítása	14
3.3.1. Leendő lokális pozíció meghatározása	14
3.3.2. Megszorítások alkalmazása	14
4. Ütközésetektálás	15
4.1. Széles fázis	15
4.2. Szűk fázis	15
4.3. További optimalizáció	16
5. SAT	17
5.1. Tengelyek meghatározása	17
5.2. Ütközés megállapítása	18
5.3. Átfedés mértékének meghatározása	18
5.4. Ütközésnormál előállítás	18
6. Ütközésfeloldás	20
6.1. Pozíciókorrigálás	20

6.2. Sebességmódosítás	20
7. Adatbázis	22
Felhasználói dokumentáció	22
Csapatmunka részletei	28
Összefoglalás	29
Irodalomjegyzék	31

Bevezetés

Téma ismertetése

Az applikáció egy 2D-s JavaScript játék megvalósítása. A felhasználó egy űrhajó kapitánya. A játékmenet során a játékoskal szembeszálló ellenséges űrjárművek legyőzése a cél, melyet a játék tapasztalatpontokkal (XP) jutalmaz. A játék online felülete lehetővé teszi a felhasználó számára a barátok keresését, barátként való bejelölésüket, illetve ezen barátok közötti versenyek indítását. A versenyek célja, hogy egy adott határidőig a lehető legtöbb tapasztalatpontot szerezzék a játékosok.

Szakmai cél

A valós idejű játékok fejlesztése komoly kihívásokat állít a fejlesztők elé, elsősorban matematikai és optimalizálási szempontból. Mindannyian kíváncsiak voltunk, hogy hogyan épülnek fel az ilyen játékok, és szerettük volna első kézből megtapasztalni, valamint megoldani ezeket a kihívásokat. Ezen 2D-s játék fejlesztése során lehetőségünk nyílt megismerkedni a játékkészítés technikáival, és biztos alapot szerezni a fejlesztés alapvető folyamataiban.

Témaválasztás indoklása

A projekt Csongrádi Richárd játékkészítés iránti lelkesedését és Sólyom Krisztián rakéták iránti szenvedélyét tükrözi.

Fejlesztői dokumentáció

1. A játék alapjai

1.1. A szimuláció frissítési üteme

A játéklogika frissítését kétféleképpen valósíthatjuk meg:

1. **Fix időlépésű szimuláció** - a logika minden frissítése azonos időközönként történik.
2. **Változó időlépésű szimuláció** - a frissítés az eltelt valós időhöz igazodik.

A gyakorlatban a legtöbb motor kombinált megközelítést alkalmaz: a fizika fix időlépésben, a megjelenítés (render) pedig változó időlépésben fut, így mindkét terület optimális működése biztosított.

A fizika fix időlépésenkénti frissítése garantálja a szimuláció determinisztikus viselkedését.

1.2. Animálás böngészőben

Mivel böngészős játékról beszélünk így muszáj említést tenni a *window* objektum *requestAnimationFrame* (*rAF*) metódusáról.

A *rAF* egy a fejlesztő által megadott callback függvényt hív meg. A függvényhívásokat a böngésző a kijelző frissítési frekvenciájához igyekszik igazítani, így ideális animációkhoz. Használatának egyik nagy előnye, hogy a *rAF* hívások automatikusan szünetelnek, ha a lap nem látható vagy a böngésző a háttérben van, ezzel csökkentve a CPU/GPU terhelést, és ezáltal hordozható eszközök esetén növelve az akkumulátor élettartamát.

A fentiek alapján a játékunk alap infrastruktúrája egyszerűen meghatározható: a fizikai szimuláció fix időlépésenként, a vizuális megjelenítés pedig változó időlépésenként történik, mindezek a *rAF* használatával szinkronizálva.

2. Merev testek fizikai jellemzői

A merev testek azon tulajdonsága, hogy bármely rajtuk választott két pont távolsága állandó marad, függetlenül a testre ható erőktől, különösen előnyössé teszi a játékban való alkalmazásukat.

A mi esetünkben ezek a merev testek összetett objektumok, vagyis kisebb merev testek alkotják őket, melyek egymáshoz viszonyított pozíciójuk állandó.

Most ismertetésre kerül, hogyan lehet meghatározni ezeknek a testeknek bizonyos fizikai jellemzőit, melyek szükségesek a későbbi számításokhoz.

2.1. Lineáris tehetetlenség

A test lineáris tehetetlensége megegyezik a test össztömegével. Mivel a mi esetünkben a testeket kisebb objektumok alkotják, melyek tömege ismert, így ezek összegzésével megkapjuk az összetett test tehetetlenségét.

Egy n darab objektumból álló merev test esetében ez a következőképpen határozható meg:

$$M = \sum_{i=1}^n m_i,$$

ahol m_i az összetett testet alkotó i -edik objektum tömege, M pedig az összetett merev test tömege.

2.2. Tömegközéppont

A tömegközéppont egy rendszer azon pontja, melynek mozgása jellemzi az egész rendszer mozgását. Emellett megállapítása elengedhetetlen ha valósághűen szeretnénk modellezni egy test forgását. A következő képlet segítségével megállapítható egy rendszer, vagy összetett test tömegközéppontja:

$$\mathbf{G} = \frac{1}{M} \sum_{i=1}^n m_i (\mathbf{r}_i + \mathbf{G}_i),$$

ahol m_i az i -edik objektum tömege, $\mathbf{r}_i$ az i -edik objektum lokális pozíciója, $\mathbf{G}_i$ az i -edik objektum tömegközéppontja, M pedig az összetett test teljes tömege.

2.3. Tehetetlenségi nyomaték

A tehetetlenségi nyomaték (I) azt fejezi ki, hogy egy test mennyire áll ellen a szögsebességében bekövetkező változásoknak, vagyis mennyire nehéz megváltoztatni annak forgását egy adott tengely körül.

Egy testnek többféle tehetetlenségi nyomatéka is lehet, hisz azok relatívak egy adott tengelyhez viszonyítva.

2.3.1. Párhuzamos tengely tétel

A tény, hogy egy test tehetetlenségi nyomatéka minden tengely esetén más elsőre nagyon bonyolultnak érezteti a témát. Itt segítségünkre siet a "párhuzamos tengely tétel", mely segítségével bármilyen tengelyre meghatározható a test tehetetlenségi nyomatéka, ha az ismert a test tömegközéppontján átmenő tengelyhez viszonyítva.

A tehetetlenségi nyomaték így tehát a következőképpen számítható ki:

$$I = I_{\text{cm}} + md^2,$$

ahol I a kívánt tengelyhez viszonyított tehetetlenségi nyomaték, I_{cm} a test tehetetlenségi nyomatéka a tömegközéppontján átmenő tengely körül, m a test tömege, d pedig a két tengely távolsága.

2.3.2. Tetszőleges sokszög tehetetlenségi nyomatéka

A merev testünket ilyen sokszögek alkotják, vagyis a teljes test tehetetlenségi nyomatékának meghatározásához először az azt alkotó kisebb objektumokét kell meghatározni. Ezek az értékek pedig a későbbiek során a "párhuzamos tengely tétel" segítségével felhasználhatóak lesznek bármilyen más tengelyhez viszonyítva.

Célszerű tehát ezen sokszögek tehetetlenségi nyomatékát a tömegközéppontjukhoz viszonyítva meghatározni. Ehhez, nem meglepő módon meg kell határoznunk azok tömegközéppontját, illetve területét. Erre egy könnyen algoritmizálható megoldást nyújt a cipőfűző képlet (shoelace formula).

Egy $n - 1$ csúcsú sokszög területe:

$$A = \frac{1}{2} \left| \sum_{i=1}^{n-1} x_i y_{i+1} - x_{i+1} y_i \right|, \quad (1)$$

tömegközéppontja:

$$\mathbf{G} = \frac{1}{6A} \begin{bmatrix} \sum_{i=1}^{n-1} (x_i + x_{i+1}) \cdot (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i) \\ \sum_{i=1}^{n-1} (y_i + y_{i+1}) \cdot (x_i \cdot y_{i+1} - x_{i+1} \cdot y_i) \end{bmatrix},$$

ha csúcsai:

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}, \dots, \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}, \begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Most, hogy meghatároztuk a sokszög területét és tömegközéppontját meghatározhatjuk annak tehetetlenségi nyomatékát a következő módon:

$$I = \frac{m}{6A} \sum_{i=1}^{n-1} (x_i'^2 + y_i'^2 + x_i' x_{i+1}' + y_i' y_{i+1}' + x_{i+1}'^2 + y_{i+1}'^2) (x_i' y_{i+1}' - x_{i+1}' y_i'),$$

(1): az abszolútérték jelek elhagyhatók, ha a sokszög csúcsai az óramutató járásával ellentétes irányba vannak rendezve.

ha $x'_i = x_i - \mathbf{G}_x$ és $y'_i = y_i - \mathbf{G}_y$, ahol i a sokszög csúcsait indexeli.

2.3.3. Összetett test tehetetlenségi nyomatéka

Az összetett testet alkotó objektumok tehetetlenségi nyomatékát és lokális pozícióját ismerve egy n objektumot tartalmazó összetett merev test tehetetlenségi nyomatéka a következő módon állapítható meg:

$$I = \sum_{i=1}^n I_i + m_i \|\mathbf{r}_i - \mathbf{G}\|_2^2,$$

ahol G az összetett test tömegközéppontja.

3. Játékmechanika

3.1. Rakétahajtóművek

Mivel játékunk a világűrben, rakétákkal zajlik, a rakétahajtóművek hozzáadása természetes és nélkülözhetetlen eleme a játéknak. A valóságban az ilyen hajtóművek végtelenül komplex szerkezetek, így annak érdekében, hogy a játék élvezhető legyen és ne igényeljen átfogó tudást a témában némi egyszerűsítést eszközöltünk.

3.1.1. Lineáris gyorsulás

Egy test akkor tapasztal lineáris gyorsulást, ha a rá ható erők összege nem nulla.

$$\mathbf{F} \neq \mathbf{0}, \text{ ahol } \mathbf{F} = \sum_i \mathbf{F}_i$$

Ahhoz tehát, hogy meghatározzuk az adott test gyorsulását, összegezni kell az egyes hajtóművek által generált tolóerőt.

Amint ez megtörtént a gyorsulás a következő képlet segítségével határozható meg:

$$\mathbf{a} = \frac{\mathbf{F}}{m},$$

ahol $\mathbf{F}$ a testre ható erők összege, m pedig a test tömege. A gyorsulás nagysága $\frac{\|\mathbf{F}\|_2}{m}$, iránya pedig megegyezik $\mathbf{F}$ irányával.

3.1.2. Tolóerő

A tolóerő a hajtóműből kiáramló nagysebességű gázok által a rakétára generált ellenerő, melynek meghatározására számos képlet áll rendelkezésünkre. Implementációnkban a hajtóművek fajlagos impulzusa (I_{sp}) és tömegárama ($\dot{m}$) adott. Ezek tekintetében a tolóerő a következőképpen határozható meg:

$$\mathbf{T} = \mathbf{v}_e \cdot \dot{m},$$

ahol $\mathbf{v}_e$ a hajtóműből kiáramló gáz sebessége, mely a következő módon számítható ki:

$$\mathbf{v}_e = I_{sp} \cdot g_0,$$

ahol g_0 a gravitáció által kifejtett gyorsulás a Föld felszínén ($\sim 9,81m/s^2$).

3.1.3. Forgatónyomaték

A forgató nyomaték kiszámítása a következő képlettel történik:

$$\mathcal{T} = \mathbf{r} \times \mathbf{F},$$

ahol $\mathbf{F}$ a hajtómű által kifejtett erő, $\mathbf{r}$ pedig a forgástengelytől az erő hatáspontjáig mutató vektor.

A képletből a következő megállapítás vonható le:

$$\mathbf{F} \parallel \mathbf{r} \Leftrightarrow \mathcal{T} = 0,$$

vagyis ha az erő párhuzamos az $\mathbf{r}$ helyvektorral, akkor az nem járul hozzá a testre ható forgatónyomatékhoz.

Mivel kétdimenziós térben vagyunk, így az $\mathbf{r}$ és $\mathbf{F}$ vektorokat fel kell írunk háromdimenziós alakra. Ez a következőképpen néz ki:

$$\mathbf{r} = \begin{bmatrix} \mathbf{r}_x \\ \mathbf{r}_y \\ \mathbf{r}_z \end{bmatrix} \text{ és } \mathbf{F} = \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \\ \mathbf{F}_z \end{bmatrix},$$

ahol x és y mindkét esetben az eredeti kétdimenziós vektor x és y értéke, míg $z = 0$, így a vektoralis szorzat értelmezhető a két, immár háromdimenziós vektor között. A nyomatékot tehát a következő képlet adja meg:

$$\mathcal{T} = \begin{bmatrix} \mathbf{r}_y \mathbf{F}_z - \mathbf{r}_z \mathbf{F}_y \\ \mathbf{r}_z \mathbf{F}_x - \mathbf{r}_x \mathbf{F}_z \\ \mathbf{r}_x \mathbf{F}_y - \mathbf{r}_y \mathbf{F}_x \end{bmatrix}, \text{ viszont mivel } \mathbf{r}_z \text{ és } \mathbf{F}_z = 0 \Rightarrow \mathcal{T} = \begin{bmatrix} 0 \\ 0 \\ \mathbf{r}_x \mathbf{F}_y - \mathbf{r}_y \mathbf{F}_x \end{bmatrix}$$

A képlet tehát kétdimenziós kiindulási vektorok esetén egy egyszerűbb formában is felírható, miképpen a nyomaték ($\mathcal{T}$) egy skalár lesz:

$$\mathcal{T} = \mathbf{r}_x \mathbf{F}_y - \mathbf{r}_y \mathbf{F}_x$$

3.1.4. Szöggyorsulás

Egy kétdimenziós test szöggyorsulása a következőképpen számítható ki:

$$\alpha = \frac{1}{I} \sum_i \mathcal{T}_i,$$

ahol α a szöggyorsulás, $\mathcal{T}_i$ az i -edik hajtómű által generált forgatónyomaték, I pedig a test tehetetlenségi nyomatéka.

3.1.5. Szögsebesség

Egy kétdimenziós test szögsebessége a következőképpen adható meg:

$$\omega = \omega_0 + \alpha \cdot t,$$

ahol a test végső szögsebessége ω , kezdeti szögsebessége ω_0 , szöggyorsulása pedig α .

3.2. Lövés

Egy lövöldözős játék esetében elengedhetetlen, hogy mind a játékos, mind pedig az őt támadó ellenséges karakterek rendelkezzenek a lövedékek valamilyen formában történő kibocsátásának képességével. Ennek implementálása attól függően, hogy a játékos vagy az ellenséges karakterek számára történik nagyban eltér.

3.2.1. Játékos lövésmechanikája

A játékos esetében nincs nehéz dolgunk. Itt teljesen elfogadható, hogy a lövedék arra megamerre a játékos nézett, így némi kihívással megfűszerezve a játékot.

3.2.2. Ellenségek lövésmechanikája

Az ellenségek esetében kissé változik a helyzet. A felmerülő probléma az, hogy ha az ellenségek egy mozgó játékos esetében a játékos konkrét pozíciójára céloz, akkor semmi esélye nincs eltalálni azt. A játék ennek köszönhetően semmilyen kihívást nem fog nyújtani a felhasználó számára.

A megoldást az úgynevezett *előtartás* nyújtja.

3.2.3. Előtartás

A probléma tehát adott: a játékos jelenlegi, világbeli pozíciója ($\mathbf{r}_{\text{játékos}}$), sebességének iránya és nagysága ($\mathbf{v}_{\text{játékos}}$), illetve a lövedék kiinduló, világbeli pozíciója ($\mathbf{r}_{\text{lövedék}}$) és sebességének nagysága ($\mathbf{v}_{\text{lövedék}}$) ismeretében meg kell találni azt a $t \geq 0$ időpontot, amikor a lövedék és a célpont ugyanabban a pontban lesznek.

Az megoldandó egyenlet tehát a következő:

$$\mathbf{r}_{\text{játékos}} + \mathbf{v}_{\text{játékos}} \cdot t = \mathbf{r}_{\text{lövedék}} + \mathbf{v}_{\text{lövedék}} \cdot t$$

egy kis átrendezés:

$$\mathbf{r} = (\mathbf{v}_{\text{lövedék}} - \mathbf{v}_{\text{játékos}}) \cdot t, \text{ ahol } \mathbf{r} = \mathbf{r}_{\text{játékos}} - \mathbf{r}_{\text{lövedék}}$$

$$\mathbf{r} = (\mathbf{v}_{\text{lövedék}} - \mathbf{v}_{\text{játékos}}) \cdot t \Rightarrow \mathbf{v}_{\text{lövedék}} = \mathbf{v}_{\text{játékos}} + \frac{\mathbf{r}}{t}$$

mivel a lövedék sebességének csak a nagyságát ismerjük:

$$\|\mathbf{v}_{\text{lövedék}}\|_2 = s$$

helyettesítve:

$$\|\mathbf{v}_{\text{játékos}} + \frac{\mathbf{r}}{t}\|_2 = s$$

mindkét oldalt szorozva t-vel:

$$\|\mathbf{r} + \mathbf{v}_{\text{játékos}} \cdot t\|_2 = s \cdot t$$

a gyökjel eltüntetéséhez négyzetre emelünk:

$$\|\mathbf{r} + \mathbf{v}_{\text{játékos}} \cdot t\|_2^2 = (s \cdot t)^2 \Rightarrow (\mathbf{r} + \mathbf{v}_{\text{játékos}} \cdot t) \cdot (\mathbf{r} + \mathbf{v}_{\text{játékos}} \cdot t) = s^2 t^2$$

felbontva:

$$\mathbf{r}^2 + 2(\mathbf{r} \cdot \mathbf{v}_{\text{játékos}})t + (\mathbf{v}_{\text{játékos}} \cdot \mathbf{v}_{\text{játékos}})t^2 = s^2 t^2$$

nullára rendezve:

$$\|\mathbf{v}_{\text{játékos}}\|_2^2 t^2 - s^2 t^2 + 2(\mathbf{r} \cdot \mathbf{v}_{\text{játékos}})t + \|\mathbf{r}\|_2^2 = 0 \Leftrightarrow (\|\mathbf{v}_{\text{játékos}}\|_2^2 - s^2)t^2 + 2(\mathbf{r} \cdot \mathbf{v}_{\text{játékos}})t + \|\mathbf{r}\|_2^2 = 0$$

az egyenlet megoldásai:

- $a = \|\mathbf{v}_{\text{játékos}}\|_2^2 - s^2$
- $b = 2(\mathbf{r} \cdot \mathbf{v}_{\text{játékos}})$
- $c = \|\mathbf{r}\|_2^2$

$$t_1, t_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Ha $t > 0$, akkor a lövedék sebességének irányát a következőképpen kaphatjuk meg:

$$\mathbf{d} = (\mathbf{v}_{\text{játékos}} \cdot t + \mathbf{r}_{\text{játékos}}) - \mathbf{r}_{\text{lövedék}}$$

az irányvektor tehát: $\hat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|_2}$

3.3. Játékosmodell módosítása

Játékunkban a felhasználónak lehetősége van űrhajójának modelljét módosítani blokkok lecsatolásával vagy hozzáadásával. Ehhez egerével meg kell ragadnia az űrben lebegő gazdátlan egységeket, majd hajójához húznia mindaddig míg nem kerül kontaktusba a blokkal.

3.3.1. Leendő lokális pozíció meghatározása

Amint az eger irányítása alatt álló blokk ütközik a játékos űrhajójával a játék meghatároz neki egy leendő, a modellhez lokális pozíciót a következő módon:

$${}^L\mathbf{r}_{\text{blokk}} = \lfloor {}^V\mathbf{r}_{\text{blokk}} - {}^V\mathbf{r}_{\text{játékos}} \rfloor,$$

ahol ${}^L\mathbf{r}$ az adott entitás lokális pozíciója, ${}^V\mathbf{r}$ pedig az annak világbeli pozíciója. A meghatározott pozíció ezután ellenőrzésre kerül, hogy megfelel-e minden megszorításnak.

3.3.2. Megszorítások alkalmazása

A szóban forgó modellmódosításokra természetesen megszorításokat alkalmaztunk, melyek a következők:

- A csatolásra kerülő blokk nem helyezhető fel úgy, hogy legalább az egyik oldalán ne érintkezzen egy már a modellben lévő blokkal.
- Olyan blokk nem távolítható el, melynek nincs legalább egy szabad oldala.

Az első megszorítás biztosításához a következő feltételnek kell teljesülnie:

$$\{\text{blokk} \in M \mid \|\mathbf{r}_{\text{hozzáadandó}} - \mathbf{r}_{\text{blokk}}\|_1 = 1\} \neq \emptyset \wedge \{\text{blokk} \in M \mid \mathbf{r}_{\text{hozzáadandó}} = \mathbf{r}_{\text{blokk}}\} = \emptyset,$$

ahol M a játékosmodell blokkjainak halmaza, $\mathbf{r}_{\text{hozzáadandó}}$ a hozzáadni kívánt blokk leendő lokális pozíciója, $\mathbf{r}_{\text{blokk}}$ pedig a modell egyik blokkjának a modellhez lokális pozíciója.

Blokkok eltávolítása esetén megvizsgálásra kerül az eltávolítandó elem négy oldala. Amennyiben legalább az egyik nincs fedésben a modell egy másik eleme által a blokk lecsatlakoztatható a játékosmodellről. A négy vizsgálandó irány esetén

$$\hat{\mathbf{d}}_{\uparrow} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \hat{\mathbf{d}}_{\rightarrow} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \hat{\mathbf{d}}_{\downarrow} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \hat{\mathbf{d}}_{\leftarrow} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$$

a következő feltételnek kell teljesülnie, hogy a blokk eltávolítható legyen, ha $\mathbf{r}_{\text{eltávolítandó}}$ az eltávolítandó blokk lokális pozíciója:

$$\exists i \in \{\uparrow, \rightarrow, \downarrow, \leftarrow\}: \{\text{blokk} \in M \mid \mathbf{r}_{\text{eltávolítandó}} + \hat{\mathbf{d}}_i = \mathbf{r}_{\text{blokk}}\} = \emptyset$$

4. Ütközésdetektálás

Az ütközésdetektálás (collision detection) a játékok és fizikai szimulációk egyik alapvető rendszere, amely meghatározza, hogy két objektum érintkezik-e, metszi-e egymást, vagy közvetlenül közelednek-e oly módon, hogy interakció lépjen fel közöttük.

Feladata felismerni minden olyan helyzetet, ahol két modell, hitbox vagy fizikai test átfedésbe kerül, illetve ahol az átfedés a következő lépésben bekövetkezhet.

A rendszer tipikusan két szakaszra oszlik: széles fázisra (broad phase), amely gyorsan szűri a potenciálisan ütköző objektumokat, és szűk fázisra (narrow phase), amely részletesen vizsgálja a valódi geometriai metszéseket.

4.1. Széles fázis

A széles fázis célja, hogy gyorsan kiszűrje a biztosan nem ütköző objektumokat, és azonosítsa azokat a potenciálisan ütköző párokat, amelyeket a szűk fázisban részletesebben kell vizsgálni. Ezzel a megközelítéssel elkerülhető, hogy a szűk fázis, amely jóval összetettebb és erőforrásigényesebb algoritmusokat használ, feleslegesen dolgozzon olyan objektumokon, amelyekről már a széles fázis könnyű ellenőrzése megállapítja, hogy nem ütközhetnek egymással.

A mi megvalósításunkban az objektumok széles fázisú ütközésvizsgálatához az objektum köré írható legkisebb sugarú kör szolgál ütközőtesttül.

A végrehajtandó ellenőrzés tehát a következő:

$$\|\mathbf{r}_a - \mathbf{r}_b\|_2 < R_a + R_b$$

vagy

$$(\mathbf{r}_a - \mathbf{r}_b) \cdot (\mathbf{r}_a - \mathbf{r}_b) < (R_a + R_b)^2 \quad (2)$$

ahol $\mathbf{r}_a$ és $\mathbf{r}_b$ a két objektum világbeli pozíciója, R_a és R_b pedig a két objektum köré írható legkisebb sugarú kör sugara.

Ha a kifejezés logikai értéke igaz, a két objektumot potenciálisan ütköző párnak tekintjük; ha hamis, akkor biztosan nem ütközhetnek.

4.2. Szűk fázis

A szűk fázis során pontosan meghatározásra kerül, hogy az adott objektum-pár ütközik-e, vagy nem. Ennek meghatározására két elterjedt megoldás létezik: a SAT (Separating Axis Theorem) és a GJK (Gilbert-Johnson-Keerthi) algoritmusok. A SAT algoritmus kezdőbarátabb és sok esetben optimálisabb megoldás is, így mi ezen algoritmus használata mellett döntöttünk. A későbbiekben ezen algoritmus működése lesz ismertetve.

Itt már az ütközőtest nem feltétlen egy kör, hanem egy olyan alakzat, amely pontosan körbehatárolja a kívánt területet. Ez az alakzat egyszerűbb platformer játékok esetében egy AABB

(2): az egyenlet megegyezik az őt megelőző formulával, azonban mivel nem tartalmaz vektorhossz-számítást, négyzetgyökmentes formában van, ami számítási szempontból hatékonyabb.

(Axis-Aligned Bounding Box), szabadabb mozgást megengedő játékok esetében pedig általában egy OBB (Oriented Bounding Box). A mi esetünkben a szűk fázisú ütközőtestek bármilyen OBP-k (Oriented Bounding Polygon) lehetnek, a lehető legnagyobb alkotói szabadság megengedése érdekében.

4.3. További optimalizáció

Az ütközésdetektálás széles és szűk fázisra való felosztásán felül további lehetőségek is rendelkezésünkre állnak, melyekkel tovább csökkenthetjük a rendszer terheltségét. Ezek közül csak egyet alkalmaztunk, így csak azt fogjuk megemlíteni.

Az egyenletes térbeli felosztású rács (Uniform Spatial Subdivision Grid) alkalmazásával a világot egyenlő méretű cellákra osztjuk. Ezekbe a cellákba aztán később regisztráljuk azon entitásokat, melyeket teljesen vagy részlegesen tartalmaznak. Csak azok az objektumok ütközhetnek, amelyek közös cellán osztoznak, így csak az egy cellában lévő objektumokat kell ellenőrizni egymással. Az előny magától értetődő: az egymástól távol lévő objektumok nem ütközhetnek, és a rács használatával még a széles fázis könnyű algoritmusának futtatását is megspóroltuk.

5. SAT

A SAT (Separating Axis Theorem) az ütközésérzékelés szűk fázisa során használt algoritmus, matematikai tétel, mely a következőt mondja ki:

Ha két alakzat között található olyan tengely, amelyre vetítve azok nem metszik egymást, akkor biztosan kimondható, hogy a két alakzat nincs átfedésben.

5.1. Tengelyek meghatározása

Első lépésként a tengelyek meghatározása a feladatunk. AABB ütközőtestek esetén ez könnyű, hiszen a jól ismert x és y tengelyen kívül nincs is másra szükségünk. Amennyiben azonban ütközőtestek tetszőleges számú oldallal rendelkeznek, illetve oldalaik nem párhuzamosak, vagy merőlegesek az x és y tengelyre a dolgunk már kissé nehezebb.

Egy tetszőleges $n - 1$ darab csúccsal rendelkező sokszögből a következő képpen állíthatóak elő a szükséges tengelyek, ha a sokszög csúcsai

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}, \dots, \begin{bmatrix} x_{n-1} \\ y_{n-1} \end{bmatrix}, \begin{bmatrix} x_n \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

akkor az i -edik élvektor

$$\mathbf{e}_i = \begin{bmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \end{bmatrix}$$

az i -edik élvektor normálvektora

$$\mathbf{n}_i = \mathbf{e}_i \cdot \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \text{ vagy } \mathbf{n}_i = \mathbf{e}_i \cdot \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

a tengelyek tehát:

$$A_a = \{\mathbf{n}_i \mid i = 1, \dots, n - 1\},$$

ahol A_a a tetszőleges a sokszögből előállított tengelyek halmaza.

A vizsgálandó tengelyek tehát a fentebb látható módon előállíthatóak mindkét sokszög csúcsaiból. Az összes vizsgálandó tengely a és b objektumok esetén:

$$A = A_a \cup A_b,$$

ahol A az összes vizsgálandó tengely halmaza, A_a az egyik sokszögből előállított tengelyek halmaza, A_b pedig a másik sokszög tengelyeinek halmaza.

5.2. Ütközés megállapítása

Következő feladatunk a két sokszög összes csúcsát rávetíteni az előzőleg definiált tengelyekre. Minden tengely esetén nyomon kell követni az adott sokszög csúcsainak vetítési minimum és maximum értékét. Amint végeztünk egy tengelyre való vetítéssel és megkaptuk a végleges minimum és maximum értékeket a következő reláció teljesülése esetén a két alakzat nem fedi egymást:

$$a_{\max} \leq b_{\min} \vee b_{\max} \leq a_{\min}$$

A fentebb látható reláció teljesülése esetén tehát a tengelyek további vizsgálata fölösleges, hisz biztosan megállapítható, hogy a két alakzat nincs átfedésben.

A tengelyre vetítés a következőképpen történik:

$$\mathbf{P}_j \cdot \mathbf{n}_i,$$

ahol $\mathbf{P}_j$ az adott sokszög j -edik csúcsa, $\mathbf{n}_i$ pedig az i -edik tengely. Ezen skaláris szorzat eredménye az, amelyet aztán össze kell hasonlítani a jelenleg legkisebbnek és legnagyobbnak tartott eredménnyel és megfelelően frissíteni azokat.

5.3. Átfedés mértékének meghatározása

Az átfedés mértéke a következő képlettel adható meg:

$$\min(a_{\max}, b_{\max}) - \max(a_{\min}, b_{\min}),$$

Ez az érték később használható a két objektum szétválasztására, vagyis az átfedés megszüntetésére.

5.4. Ütközésnormál előállítása

Az ütközés normálja (collision normal), vagyis az a tengely, vagy irány, mely mentén a két objektum szétválasztandó a vizsgált tengelyek közül az, amelyiken a két objektum átfedése a legkisebb volt.

Egyszerű, vagyis olyan testek esetén melyek ütközésében csakis két ütközőtest vesz részt ennek meghatározása könnyű: azt a tengelyt keressük, amelyen az ütköző objektumok átfedése a legkisebb volt. Komplex testek esetén, mint például a konkáv alakzatok, melyek esetén két test ütközése több kisebb alütközéssel jár ezen normál megállapítása kissé bonyolultabbá válik. Ez esetben a leghatékonyabbnak bizonyuló megoldás a főnormál, vagyis azon normál előállítására mely az ütközésfeloldás során használatos az alütközések normálisának összegzése.

$$\mathbf{N} = \sum_i \mathbf{n}_i,$$

ahol $\mathbf{N}$ a főnormál, $\mathbf{n}_i$ pedig az i -edik alütközés normálisa.

Ezzel a megoldással a konkáv alakzatok esetén - melyek konvex dekompozícióra kényszerülnek - az ütközések és azok feloldása ugyanolyan simán, egyenletesen és akadózásmentesen történik, mint az egyszerű testeké.

6. Ütközésfeloldás

Az ütközésfeloldás során az átfedésben lévő alakzatokat szétválasztjuk pozíciójuk módosításával, lineáris és szögsebességük pedig az elérni kívánt fizikai valóságszerűség függvényében módosításra kerülnek.

6.1. Pozíciókorrigálás

Az ütközésben résztvevő objektumok szétválasztásával, vagyis pozíciójuk módosításával az ütközés megszüntetésre kerül. Ehhez szükségünk van az ütközés normáljára, mint irányra ami mentén az objektumokat mozgatjuk, illetve az átfedés mértékére, amivel az objektumok pozícióját módosítjuk.

Fontos, hogy az ütközés normálját normalizáljuk, ha még nem tettük:

$$\hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|_2}$$

Ha $\mathbf{a}$ és $\mathbf{b}$ objektumok és x az átfedés mértéke, akkor a módosított pozíciójuk a következőképpen adható meg:

$$\mathbf{r}'_a = \mathbf{r}_a - (x + \varepsilon) \cdot \frac{m_a}{m_a + m_b} \cdot \hat{\mathbf{n}}$$

és

$$\mathbf{r}'_b = \mathbf{r}_b + (x + \varepsilon) \cdot \frac{m_b}{m_a + m_b} \cdot \hat{\mathbf{n}},$$

ahol $\mathbf{r}$ az adott objektum világbeli pozíciója, m a tömegük, ε pedig egy kis érték, mint például 0,001, mely hozzáadásával a lebegőpontos számábrázolás pontatlansága nem okoz "akadozó", vagy "össze-vissza dobálózó" mozgást az objektumok esetében.

6.2. Sebességmódosítás

Az objektumok sebességének módosításával valósághűbbé tehetjük szimulációnkat, például azok egymásról való visszapattintásával.

Adott az $\mathbf{a}$ objektum sebessége $\mathbf{v}_a$, a $\mathbf{b}$ objektum sebessége $\mathbf{v}_b$ és az ütközés rugalmassági együtthatója $e \in [0, 1]$, amely meghatározza hogy az objektumok sebességének az ütközés normális irányával párhuzamos komponenséből mennyi marad meg (azaz, hogy az ütközés mennyire rugalmas).

Kiszámoljuk a két test relatív sebességét:

$$\mathbf{v}_r = \mathbf{v}_b - \mathbf{v}_a$$

Ha a relatív sebesség és az ütközés normálisának skaláris szorzata ($\mathbf{v}_r \cdot \hat{\mathbf{n}}$) nagyobb mint 0, akkor az azt jelenti, hogy a két test egymástól már távolodik. Ez esetben semmit sem kell tennünk.

Ezután az impulzusváltozást a következő képlettel számolhatjuk ki:

$$j = \frac{-(1+e) \cdot (\mathbf{v}_r \cdot \hat{\mathbf{n}})}{\frac{1}{m_a} + \frac{1}{m_b}}$$

Az impulzusvektor ebből:

$$\mathbf{I} = \hat{\mathbf{n}} \cdot j,$$

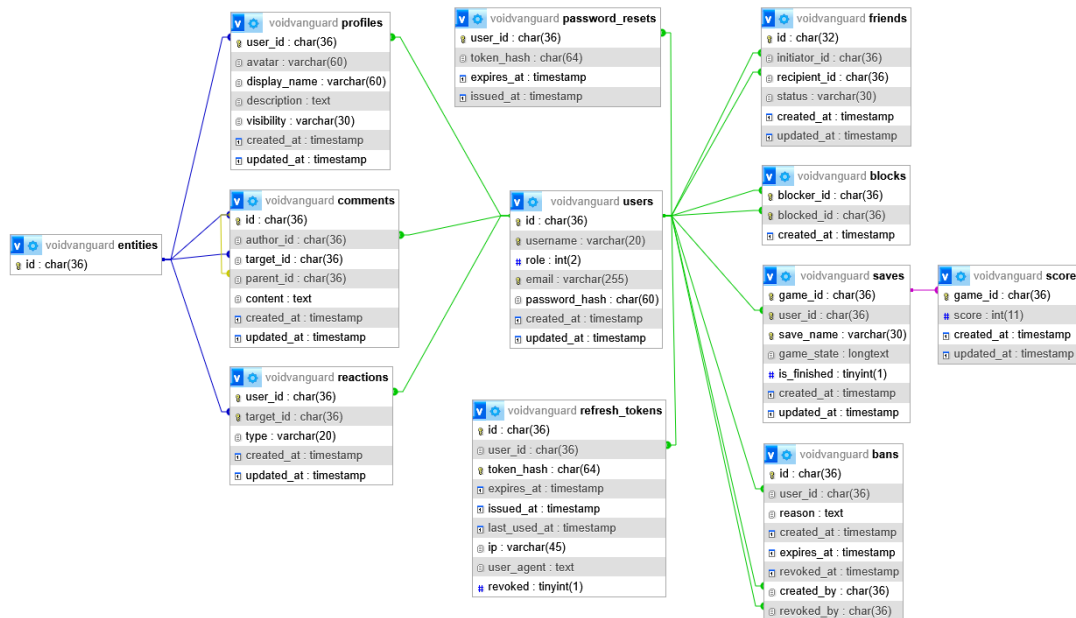
amivel a két objektum sebességét módosítanunk kell.

Az ütközésben részt vevő két objektum ütközés utáni sebességét a következőképpen kap-hajtuk meg:

$$\mathbf{v}'_a = \mathbf{v}_a - \mathbf{I} \cdot \frac{1}{m_a} \text{ és } \mathbf{v}'_b = \mathbf{v}_b + \mathbf{I} \cdot \frac{1}{m_b}$$

7. Adatbázis

Adatbázis struktúra áttekintése



Mező	Típus	Korlátok	Leírás
id	CHAR(36)	PRIMARY KEY	Egyedi UUID azonosító
username	VARCHAR(20)	UNIQUE, NOT NULL	Felhasználó neve
role	INT(2)	DEFAULT 0, NOT NULL	Felhasználó jogosultságát jelző érték
email	VARCHAR(255)	UNIQUE, NOT NULL	Email cím
password_hash	CHAR(60)	NOT NULL	Felhasználó jelszava hashelve
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

1. táblázat. Felhasználók táblája

Mező	Típus	Korlátok	Leírás
user_id	CHAR(36)	PRIMARY KEY, FOREIGN KEY	A birtokló felhasználó azonosítója
avatar	VARCHAR(60)	DEFAULT NULL	Profilkép elérési útvonala
display_name	VARCHAR(60)	NOT NULL	Megjelenítési név
description	TEXT	DEFAULT NULL	Profil leírása
visibility	VARCHAR(30)	NOT NULL, DEFAULT 'public'	Profil láthatósági beállítása
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

2. táblázat. Profilok táblája

Mező	Típus	Korlátok	Leírás
id	CHAR(32)	PRIMARY KEY	A barátság kapcsolat egyedi azonosítója
initiator_id	CHAR(36)	FOREIGN KEY, NOT NULL	Kezdeményező felhasználó azonosítója
recipient_id	CHAR(36)	FOREIGN KEY, NOT NULL	Kezdeményezett felhasználó azonosítója
status	VARCHAR(30)	NOT NULL, DEFAULT 'pending'	Kapcsolat állapota
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

3. táblázat. Barátkapcsolatok táblája

Mező	Típus	Korlátok	Leírás
blocker_id	CHAR(36)	FOREIGN KEY, NOT NULL	Letiltó felhasználó azonosítója
blocked_id	CHAR(36)	FOREIGN KEY, NOT NULL	Letiltott felhasználó azonosítója
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje

4. táblázat. Letiltások táblája

Mező	Típus	Korlátok	Leírás
game_id	CHAR(36)	PRIMARY KEY	Játékmenet azonosítója
user_id	CHAR(36)	FOREIGN KEY, NOT NULL	Birtokló felhasználó azonosítója
save_name	VARCHAR(30)		Mentés neve
game_state	JSON		Játékállapot
is_finished	BOOLEAN	DEFAULT FALSE	Befejezettségi állapotot jelző mező
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

5. táblázat. Mentések táblája

Mező	Típus	Korlátok	Leírás
game_id	CHAR(36)	PRIMARY KEY, FOREIGN KEY	Játékmenet azonosítója
score	INT	NOT NULL	Pontszám
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

6. táblázat. Pontszámok táblája

Mező	Típus	Korlátok	Leírás
user_id	CHAR(36)	PRIMARY KEY, FOREIGN KEY	Felhasználó azonosítója
token_hash	CHAR(64)	NOT NULL	Jelszóvisszaállító token hashe
expires_at	TIMESTAMP	NOT NULL	Token lejárat ideje
issued_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Token kiadásának ideje

7. táblázat. Jelszóvisszaigénylő tokenek táblája

Mező	Típus	Korlátok	Leírás
id	CHAR(36)	PRIMARY KEY	Token (munkamenet) azonosítója
user_id	CHAR(36)	FOREIGN KEY, NOT NULL	Token birtokosának azonosítója
token_hash	CHAR(64)	NOT NULL, UNIQUE	Token hashelt változata
expires_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Token lejáratási időpontja
issued_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Token kiadásának időpontja
last_used_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Utolsó használat időpontja
ip	VARCHAR(45)		Tokent kérő kliens IP címe
user_agent	TEXT		Tokent kérő kliens felhasználói ügynöke
revoked	BOOLEAN	DEFAULT FALSE	Visszavonást jelző mező

8. táblázat. Refresh tokenek táblája

Mező	Típus	Korlátok	Leírás
id	CHAR(36)	PRIMARY KEY	Kitiltás azonosítója
user_id	CHAR(36)	FOREIGN KEY, NOT NULL	Kitiltás tárgyak képező felhasználó azonosítója
reason	TEXT		Indok
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
expires_at	TIMESTAMP	NULL	Kitiltás lejáratási időpontja
revoked_at	TIMESTAMP	NULL	Kitiltás visszavonási időpontja
created_by	CHAR(36)	FOREIGN KEY, NULL	Létrehozó felhasználó azonosítója
revoked_by	CHAR(36)	FOREIGN KEY, NULL	Kitiltást visszavonó felhasználó azonosítója

9. táblázat. Kitiltások táblája

Mező	Típus	Korlátok	Leírás
id	CHAR(36)	PRIMARY KEY, FOREIGN KEY	Hozzászólás azonosítója
author_id	CHAR(36)	FOREIGN KEY, NOT NULL	Szerző azonosítója
target_id	CHAR(36)	FOREIGN KEY, NOT NULL	Célpont azonosítója
parent_id	CHAR(36)	FOREIGN KEY, DEFAULT NULL	Szülő azonosítója
content	TEXT	NOT NULL	Hozzászólás tartalma
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

10. táblázat. Hozzászólások táblája

Mező	Típus	Korlátok	Leírás
user_id	CHAR(36)	PRIMARY KEY (user_id, target_id), NOT NULL	Reakciót birtokló felhasználó azonosítója
target_id	CHAR(36)	PRIMARY KEY (user_id, target_id), FOREIGN KEY	Célpont azonosítója
type	VARCHAR(20)	NOT NULL	Reakció típusa
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord beszúrásának ideje
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Rekord utolsó módosításának ideje

11. táblázat. Reakciók táblája

Mező	Típus	Korlátok	Leírás
id	CHAR(36)	PRIMARY KEY	Egyedi UUID azonosító

12. táblázat. Entitások táblája

Felhasználói dokumentáció

Csapatmunka részletei

Tövissi-Nagy Ábel

A backend fejlesztés során egy komplex közösségi rendszer került kialakításra, amely számos felhasználói funkciót integrál. Ennek részeként megvalósult a barátrendszer, a profilkezelés, valamint a kommentek és reakciók kezelése is. A rendszer tartalmaz felhasználó blokkolási lehetőséget és egy adminfelületet, amely hatékonyabbá teszi az üzemeltetést és moderációt. A biztonság érdekében kidolgozásra került a regisztrációs és bejelentkezési folyamat, token alapú hitelesítéssel. Emellett egy hangrendszer fejlesztése is megtörtént, amely növeli az alkalmazás interaktivitását. A frontend és backend közötti kommunikáció optimalizálására egy egyedi *fetch* segédfüggvény került kialakításra.

Csongrádi Richárd

A frontend fejlesztés során megvalósult egyedi design, többek között a Profilközpont, Bejelentkezési/Regisztrációs/Admin felületnél, és számos más oldalon. A játékmenet logikája valamint annak esztétikailag való megvalósulása, ebbe beleértve az űrhajó blokkjainak textúrája, a lövés és a thruster nyomvonal effektusa Canvas használatának segítségével.

A játékmenet élvezhetőségének megvalósítása, az erre szolgáló nehézségi fokozatokkal, valamint a különböző viselkedésű és felépítésű ellenséges űrhajók logikájának megírásával. Ez biztosítja, hogy a játék előrehaladtával a játékos egyre nehezebb kihívások elé kerüljön.

Sólyom Krisztián

A WebGL-renderelés implementálása biztosította a játék vizuális alapját, lehetővé téve a dinamikus elemek és környezetek részletes megjelenítését. A fizikai rendszerek, köztük az ütközésetektálás, ütközésfeloldás és rakétahajtóművek integrálása szintén a feladatok közé tartozott.

A lövésmechanika kialakítása szintén kulcsfontosságú feladat volt, mivel biztosította a megfelelő játékélményt. Ennek során a játékos és az ellenséges entitások lövésmechanikáját egyaránt implementálni kellett. Az erőalapú mozgásrendszer implementálása a karakterek fizikai mozgását és dinamikáját tette realiztikussá.

A játék reszponzivitásának megvalósítása érintőképernyős eszközök és különböző kijelzőméretek támogatásával szintén alapvető feladat volt.

A játékvilág dinamikus generálása (csillagköd háttér), amely a játék vizuális élményének fokozását szolgálja is a feladatok egyike volt.

A backend fejlesztésében egységes hibakezelés, mentési és pontszámítási végpontok, valamint adatbázis elemek készítése szerepelt. A frontend oldalon custom webkomponensek implementálása és azok tesztelése biztosította a felhasználói és backend logika közötti összekapcsolást.

Utolsó sorban pedig a fejlesztői dokumentáció elkészítése is a feladatokat sokszínűsítette.

Összefoglalás

A projekt megkezdése előtt megfogalmazott célunk – hogy új technológiákat és megközelítéseket sajátítsunk el – teljes mértékben megvalósult. A fejlesztés során nemcsak az előzetes elvárásainkat sikerült teljesíteni, hanem számos olyan területen is tapasztalatot szereztünk, amelyekre kezdetben nem is számítottunk. Ide tartozik többek között a token alapú hitelesítés alkalmazása is, amelynek megvalósítása során a modern webalkalmazások egyik alapvető biztonsági megközelítését sajátítottuk el.

A létrehozott rendszer összességében egy rendkívül komplex megoldás. Ez a komplexitás azonban nem elsősorban a funkciók számából fakad, hanem abból, hogy egy valós idejű, determinisztikus fizikára épülő szimulációt valósítottunk meg, illetve hogy már az alapoktól kezdve tudatosan törekedtünk az általánosíthatóságra. Ennek eredményeként mind a játéklógika, mind a backend architektúra olyan módon került kialakításra, hogy más, hasonló problémák megoldására is alkalmas alapot nyújthasson. További rendelkezésre álló idő esetén a rendszer funkcionalitásának bővítését, valamint a játékelmény további gazdagítását tartanánk elsődleges irányknak.

A fejlesztés során számos kihívással szembesültünk. Az egyik legmeghatározóbb ezek közül a játék alapját képező, determinisztikus fizikai szimuláció és az arra épülő vezérlési rendszer kialakítása volt, amely különösen a projekt kezdeti szakaszában bizonyult nehéznek, amikor még nem rendelkezünk számottevő tapasztalattal játékkészítés terén. Emellett jelentős kihívást jelentett a JavaScript webkomponensek életciklusának mélyebb megértése és tudatos kezelése, valamint egy olyan rugalmas rendszer megtervezése, amely biztosítja az egyes komponensek közötti kompatibilitást és bővíthetőséget.

Nehéz egyetlen területet kiemelni, ahol a legtöbbet tanultunk, azonban összességében a front-end fejlesztés bizonyult a legtanulságosabbnak. Mind a játékkészítés, mind a felhasználói felület kialakítása során számos új technológiát és szemléletmódot sajátítottunk el. Amennyiben a jövőben leginkább hasznosítható tudást tekintjük mérvadónak, akkor a webkomponensek alkalmazását emelnénk ki, mivel ez a megközelítés széles körben felhasználható modern webalkalmazások fejlesztése során. Emellett a token alapú autentikáció gyakorlati megvalósítása is olyan tapasztalatot adott, amely a későbbi backend fejlesztések során közvetlenül hasznosítható.

A projekt nemcsak egy működő játék és a hozzá kapcsolódó teljes rendszer létrehozását eredményezte, hanem egy olyan átfogó fejlesztési tapasztalatot is, amely a jövőbeni munkáink során is stabil alapot biztosít.

Irodalomjegyzék

Projekt-specifikus források

Azon források, melyek kifejezetten a projekt által nyújtott problémák megoldásának céljából lettek felkeresve.

2025. 09. 15.

- <https://chriscourses.com/blog/standardize-your-javascript-games-frame-rate-for-different-monitors>

2025. 09. 19.

- <https://www.youtube.com/watch?v=y2UsQB3WSvo>
- <https://www.youtube.com/watch?v=1La6XkVLj0w>
- <https://www.youtube.com/watch?v=sWBt5WrQTfE>
- <https://www.youtube.com/watch?v=hUxw5Ni0l78>
- <https://www.youtube.com/watch?v=C6BztStktJY>
- <https://www.youtube.com/watch?v=e5qwESME00E>

2025. 09. 21.

- <https://www.youtube.com/watch?v=yGhfUcPjXuE>
- <https://www.youtube.com/watch?v=KSbY518HNWQ>

2025. 09. 26.

- <https://www.youtube.com/watch?v=IsyHfmq18ec>
- <https://www.youtube.com/watch?v=Ude1zZbf20s>

2025. 09. 27.

- <https://www.youtube.com/watch?v=0nZn5YPNf5k>
- <https://www.youtube.com/watch?v=ocGDNM0AL3c>
- https://www.youtube.com/watch?v=R2Y6vb3z_Hw

2025. 10. 03.

- https://www.youtube.com/watch?v=w3im_9qbM18
- <https://www.youtube.com/watch?v=FCkMPkgWClo>

2025. 10. 12.

- <https://science.howstuffworks.com/rocket.htm>
- https://phys.libretexts.org/Bookshelves/University_Physics/Physics_%28Boundless%29/7%3A_Linear_Momentum_and_Collisions/7.4%3A_Rocket_Propulsion

2025. 11. 12.

-
- <https://stephendoddtech.com/blog/game-design/spiral-of-death-game-loop-javascript>

2025. 11. 12.

- https://gafferongames.com/post/fix_your_timestep/

2025. 11. 12.

- <https://peerdh.com/blogs/programming-insights/optimizing-game-loop-with-fixed-time-steps>

2025. 11. 14.

- <https://www3.ntu.edu.sg/home/ehchua/programming/howto/Regexe.html#zz-2>.

2025. 11. 15.

- <https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame>

2025. 11. 18.

- <https://www.youtube.com/watch?v=OnZn5YPNf5k>

2025. 12. 09.

- [https://phys.libretexts.org/Bookshelves/University_Physics/Physics_\(Bundless\)/7%3A_Linear_Momentum_and_Collisions/7.4%3A_Rocket_Propulsion](https://phys.libretexts.org/Bookshelves/University_Physics/Physics_(Bundless)/7%3A_Linear_Momentum_and_Collisions/7.4%3A_Rocket_Propulsion)

2025. 12. 12.

- <https://developer.mozilla.org/en-US/docs/Web/API/Node/getRootNode>

2025. 12. 18.

- <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-phy-centre-of-mass/v/center-of-mass>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-phy-centre-of-mass/v/center-of-mass-equation>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-phy-centre-of-mass/a/what-is-center-of-mass>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-phy-centre-of-mass/a/center-of-mass-ap1>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-introduction-to-angular-motion-ap/v/angular-motion-variables>
-

-
- <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-introduction-to-angular-motion-ap/v/relating-angular-and-regular-motion-variables>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-introduction-to-angular-motion-ap/a/angular-quantities-ap1>

2025. 12. 20.

- <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-angular-kinematics/v/rotational-kinematic-formulas>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-angular-kinematics/a/angular-kinematics-ap-physics-1>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/v/introduction-to-torque>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/v/moments>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/v/moments-part-2>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/v/finding-torque-for-angled-forces>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/a/torque>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-torque-and-equilibrium-ap/a/torque-and-equilibrium>
 - <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-cross-product/v/cross-product-1>
-

2025. 12. 21.

- <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-cross-product/v/cross-product-2>
- <https://www.khanacademy.org/science/in-in-class11th-physics/in-in-system-of-particles-and-rotational-motion/in-in-cross-product/v/cross-product-and-torque>

2025. 12. 29.

- <https://www.youtube.com/watch?v=5gDC1GU3Ivg>
- <https://www.youtube.com/watch?v=egmZJU-1zPU>

2025. 12. 30.

- <https://www.toptal.com/developers/game/video-game-physics-part-ii-collision-detection-for-solid-objects>
- <https://www.youtube.com/watch?v=VbvdoLQQUPs>

2025. 12. 31.

- <https://www.youtube.com/watch?v=RSXM9bgqxJM>
- <https://www.youtube.com/watch?v=VbvdoLQQUPs>

2026. 01. 01.

- https://www.chrishecker.com/Rigid_Body_Dynamics
- <https://kitemetric.com/blogs/point-in-convex-polygon-a-deep-dive>

2026. 01. 15.

- <https://www.youtube.com/watch?v=Zgf1DYrmSnk>
- <https://dyn4j.org/2010/01/sat/>
- <https://dyn4j.org/2011/11/contact-points-using-clipping/>

2026. 01. 17.

- <https://developer.mozilla.org/en-US/docs/Web/API/HTMLCanvasElement/toDataURL>
- https://www.w3schools.com/graphics/canvas_clock_numbers.asp

2026. 01. 22.

- https://www.youtube.com/watch?v=DxUY42r_6Cg
- <https://rene.ruhr/gfx/gpuhash/>

2026. 01. 23.

- <https://www.youtube.com/watch?v=ly6icfhap2o>
 - <https://sidorares.github.io/node-mysql2/docs>
-

-
- <https://www.youtube.com/watch?v=oExWh86IgHA>

2026. 01. 24.

- <https://adrianb.io/2014/08/09/perlinnoise.html>
- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/procedural-patterns-noise-part-1/introduction.html>
- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/procedural-patterns-noise-part-1/creating-simple-1D-noise.html>
- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/procedural-patterns-noise-part-1/creating-simple-2D-noise.html>
- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/procedural-patterns-noise-part-1/simple-pattern-examples.html>

2026. 01. 25.

- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/perlin-noise-part-2/perlin-noise.html>
- <https://www.scratchapixel.com/lessons/procedural-generation-virtual-worlds/perlin-noise-part-2/perlin-noise-terrain-mesh.html>
- <https://www.geeksforgeeks.org/dsa/shuffle-a-given-array-using-fisher-yates-shuffle-algorithm/>
- <https://emanueleferonato.com/2026/01/08/understanding-how-to-use-mulberry32-to-achieve-deterministic-randomness-in-javascript/>

2026. 02. 01.

- <https://www.youtube.com/watch?v=9J3UQYcro-U>

2026. 02. 02.

- <https://www.youtube.com/watch?v=4ugw5yRwhR0>
- https://www.youtube.com/watch?v=MgzVe_MJ7S8
- <https://www.youtube.com/watch?v=7Q17ubqLfaM>
- <https://www.youtube.com/watch?v=mbsmsi7l3r4>

2026. 02. 08.

- <https://www.youtube.com/watch?v=x5gLL8-M9Fo>
- https://www.youtube.com/watch?v=EIYCJKR0I_g
- <https://stackoverflow.com/questions/38552003/how-to-decode-jwt-token-without-using-a-library>

2026. 02. 12.

- <https://medium.com/@yunusemresalcan/unit-test-javascript-93ed9deb250c>

2026. 03. 16.

- https://developer.mozilla.org/en-US/docs/Web/API/Web_components
- https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_custom_elements
- https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM
- https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_templates_and_slots
- <https://developer.mozilla.org/en-US/docs/Web/API/CustomElementRegistry>

2026. 03. 19.

- https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Basic_concepts_behind_Web_Audio_API
- https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Using_Web_Audio_API
- https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Best_practices
- https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Advanced_techniques#dial-up_ăă_loading_a_sound_sample

2026. 05. 03.

- https://en.wikipedia.org/wiki/Shoelace_formula
- <https://mathworld.wolfram.com/ShoelaceFormula.html>
- https://en.wikipedia.org/wiki/Second_moment_of_area

2026. 05. 04.

- <https://chatgpt.com/>

Háttéirodalom

Szakmailag befolyásoló források, melyek közvetetten alakították a projekt során alkalmazott gondolkodásmódot és megközelítéseket.

- <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:model-situations-with-matrices/v/using-matrices-to-represent-data-payoffs>

-
- <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:matrices-as-transformations/v/matrices-as-transformations-of-the-plane>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:matrices-as-transformations/a/matrices-as-transformations>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:using-matrices-to-transform-the-plane/v/using-matrices-to-transform-the-plane-composing-matrices>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:multiplying-matrices-by-matrices/a/multiplying-matrices>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:properties-of-matrix-multiplication/a/intro-to-identity-matrices>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:representing-systems-with-matrices/v/representing-systems-of-equations-with-matrices>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:intro-to-matrix-inverses/v/invertible-matrices-and-determinants>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:intro-to-matrix-inverses/v/inverse-matrices-and-matrix-equations>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:practice-finding-inverses-of-2x2-matrices/v/inverse-of-a-2x2-matrix>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:solving-equations-with-inverse-matrices/v/solving-matrix-equation>
 - <https://www.khanacademy.org/math/prec calculus/x9e81a4f98389efdf:matrices/x9e81a4f98389efdf:solving-equations-with-inverse-matrices/a/matrices-faq>
-

- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vectors-intro/v/introduction-to-vectors-and-scalars>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vector-components/v/introduction-to-vector-components>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vec-mag/v/vector-magnitude-from-initial-and-terminal-points>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vector-add-sub/v/adding-and-subtracting-vectors>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vector-add-sub/v/visually-adding-and-subtracting-vectors>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:component-form/v/vector-component-in-direction>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vec-add-mag-dir/e/adding-vectors-in-magnitude-and-direction-form-2>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vec-models/v/vector-word-problem-resultant-force>
- <https://www.khanacademy.org/math/precalculus/x9e81a4f98389efdf:vectors/x9e81a4f98389efdf:vec-models/e/vector-word-problems>

Mindezek mellett pedig a <https://www.physicsclassroom.com/> weboldal "1-D Kinematics", "Newton's Laws", "Vectors - Motion and Forces in Two Dimensions", "Momentum and Its Conservation" és "Work and Energy" fejezeteinek tanulmányozása nélkül ezen játék ötlete fel sem merült volna.

Az ebben a szakaszban felsorolt hivatkozások utolsó megtekintési dátuma minden esetben 2026. 05. 04.